

安徽大学《深度学习与神经网络》

实验报告 6

学号： WA2224013 专业： 机器人工程 姓名： 郭义月
实验日期： 2025.6.22 教师签字： _____ 成绩： _____

[实验名称] 神经网络综合应用实验

[实验目的]

1. 熟悉和掌握利用神经网络开展视觉分类
2. 熟悉和掌握利用神经网络开展风格迁移
3. 熟悉和掌握利用神经网络开展语义分割

[实验要求]

1. 采用 Python 语言基于 PyTorch 深度学习框架进行编程
2. 代码可读性强：变量、函数、类等命名可读性强，包含必要的注释
3. 提交实验报告要求：
 - 命名方式：“学号-姓名-Lab-N” (N 为实验课序号，即：1-6)；
 - 截止时间：下次实验课当晚 23:59；
 - 提交方式：智慧安大-网络教育平台-作业；

按时提交（**过时不补**）；

[实验内容]

1. 视觉分类:

- 学习、运行、调试参考教材 13.14 小节 Kaggle 狗的品种识别
- 完成练习题 2

2. 风格迁移:

- 学习、运行和调试参考教材 13.12 小节内容
- 完成练习 1 和 2

3. 语义分割:

- 学习、运行和调试参考教材 13.9 小节内容
- 完成练习题 1

4. 参考资料:

- 参考教材: <https://zh-v2.d2l.ai/d2l-zh-pytorch.pdf>
- PyTorch 官方文档: <https://pytorch.org/docs/2.0/>;
- PyTorch 官方论坛: <https://discuss.pytorch.org/>

[实验代码和结果]

实验 1：视觉分类，学习 13.14

13.14.1 下载数据集

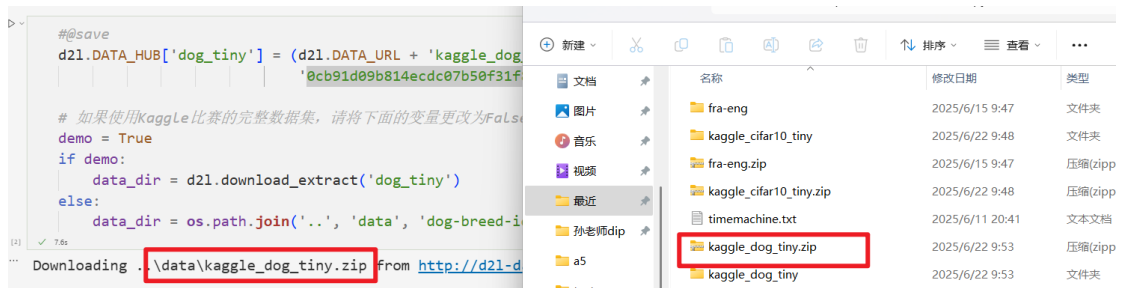
比赛数据集分为训练集和测试集，分别包含 RGB（彩色）通道的 10222 张、10357 张 JPEG 图像。在训练数据集中，有 120 种犬类，如拉布拉多、贵宾、腊肠、萨摩耶、哈士奇、吉娃娃和约克夏等。

```
#@save
d2l.DATA_HUB['dog_tiny'] = (d2l.DATA_URL + 'kaggle_dog_tiny.zip',
                             '0cb91d09b814ecdc07b50f31f8dcad3e81d6a86d')

# 如果使用Kaggle比赛的完整数据集，请将下面的变量更改为False
demo = True
if demo:
    data_dir = d2l.download_extract('dog_tiny')
else:
    data_dir = os.path.join('..', 'data', 'dog-breed-identification')
```

Downloading ..\data\kaggle_dog_tiny.zip from http://d2l-data.s3-accelerate.amazonaws.com/kaggle_dog_tiny.zip...

下载完成



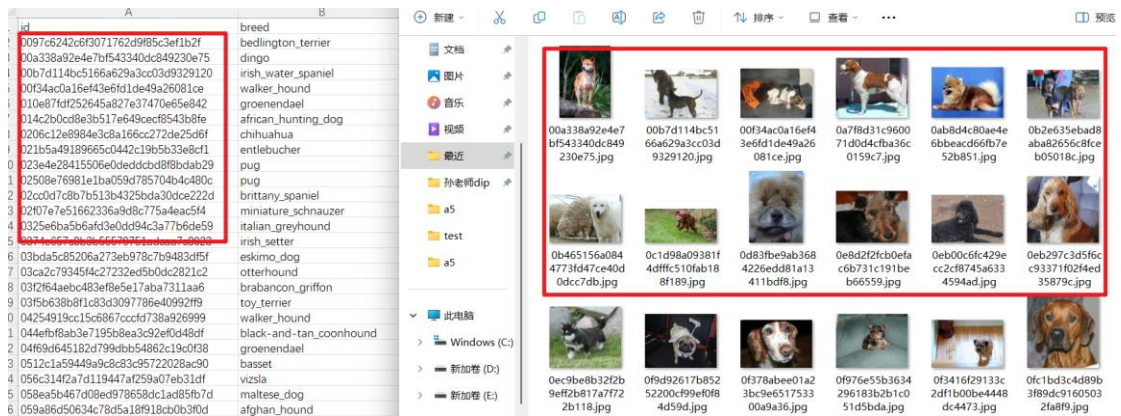
13.14.2 整理数据集

从原始训练集中拆分验证集，然后将图像移动到按标签分组的子文件夹中。

```
def reorg_dog_data(data_dir, valid_ratio):
    labels = d2l.read_csv_labels(os.path.join(data_dir, 'labels.csv'))
    d2l.reorg_train_valid(data_dir, labels, valid_ratio)
    d2l.reorg_test(data_dir)

batch_size = 32 if demo else 128
valid_ratio = 0.1
reorg_dog_data(data_dir, valid_ratio)
```

整理标签在 csv 中



13.14.3 图像增广

下图展示了如何在相对较大的图像上使用图像增广

```
transform_train = torchvision.transforms.Compose([
    # 随机裁剪图像，所得图像为原始面积的0.08~1之间，高宽比在3/4和4/3之间。
    # 然后，缩放图像以创建224x224的新图像
    torchvision.transforms.RandomResizedCrop(224, scale=(0.08, 1.0),
                                              ratio=(3.0/4.0, 4.0/3.0)),
    torchvision.transforms.RandomHorizontalFlip(),
    # 随机更改亮度，对比度和饱和度
    torchvision.transforms.ColorJitter(brightness=0.4,
                                       contrast=0.4,
                                       saturation=0.4),
    # 添加随机噪声
    torchvision.transforms.ToTensor(),
    # 标准化图像的每个通道
    torchvision.transforms.Normalize([0.485, 0.456, 0.406],
                                     [0.229, 0.224, 0.225]))])
```

测试时，我们只使用确定性的图像预处理操作。

```
transform_test = torchvision.transforms.Compose([
    torchvision.transforms.Resize(256),
    # 从图像中心裁切224x224大小的图片
    torchvision.transforms.CenterCrop(224),
    torchvision.transforms.ToTensor(),
    torchvision.transforms.Normalize([0.485, 0.456, 0.406],
                                     [0.229, 0.224, 0.225]))])
```

13.14.4 读取数据集

可以读取整理后的含原始图像文件的数据集。

```

train_ds, train_valid_ds = [torchvision.datasets.ImageFolder(
    os.path.join(data_dir, 'train_valid_test', folder),
    transform=transform_train) for folder in ['train', 'train_valid']]

valid_ds, test_ds = [torchvision.datasets.ImageFolder(
    os.path.join(data_dir, 'train_valid_test', folder),
    transform=transform_test) for folder in ['valid', 'test']]

```

创建数据加载器实例

```

train_iter, train_valid_iter = [torch.utils.data.DataLoader(
    dataset, batch_size, shuffle=True, drop_last=True)
    for dataset in (train_ds, train_valid_ds)]

valid_iter = torch.utils.data.DataLoader(valid_ds, batch_size, shuffle=False,
    drop_last=True)

test_iter = torch.utils.data.DataLoader(test_ds, batch_size, shuffle=False,
    drop_last=False)

```

13.14.5 微调预训练模型

可以使用 `numref:sec_fine_tuning` 中讨论的方法在完整 ImageNet 数据集上选择预训练的模型，然后使用该模型提取图像特征，以便将其输入到定制的小规模输出网络中。深度学习框架的高级 API 提供了在 ImageNet 数据集上预训练的各种模型。在这里选择预训练的 ResNet-34 模型，只需重复使用此模型的输出层（即提取的特征）的输入。

然后用一个可以训练的小型自定义输出网络替换原始输出层，例如堆叠两个完全连接的图层。使用三个 RGB 通道的均值和标准差来对完整的 ImageNet 数据集进行图像标准化。

```

def get_net(devices):
    finetune_net = nn.Sequential()
    finetune_net.features = torchvision.models.resnet34(pretrained=True)
    # 定义一个新的输出网络，共有120个输出类别
    finetune_net.output_new = nn.Sequential([nn.Linear(1000, 256),
                                              nn.ReLU(),
                                              nn.Linear(256, 120)])

    # 将模型参数分配给用于计算的CPU或GPU
    finetune_net = finetune_net.to(devices[0])
    # 冻结参数
    for param in finetune_net.features.parameters():
        param.requires_grad = False
    return finetune_net

```

在计算损失之前，首先获取预训练模型的输出层的输入，即提取的特征。然后使用此特征作为我们小型自定义输出网络的输入来计算损失。

```
loss = nn.CrossEntropyLoss(reduction='none')

def evaluate_loss(data_iter, net, devices):
    l_sum, n = 0.0, 0
    for features, labels in data_iter:
        features, labels = features.to(devices[0]), labels.to(devices[0])
        outputs = net(features)
        l = loss(outputs, labels)
        l_sum += l.sum()
        n += labels.numel()
    return (l_sum / n).to('cpu')
```

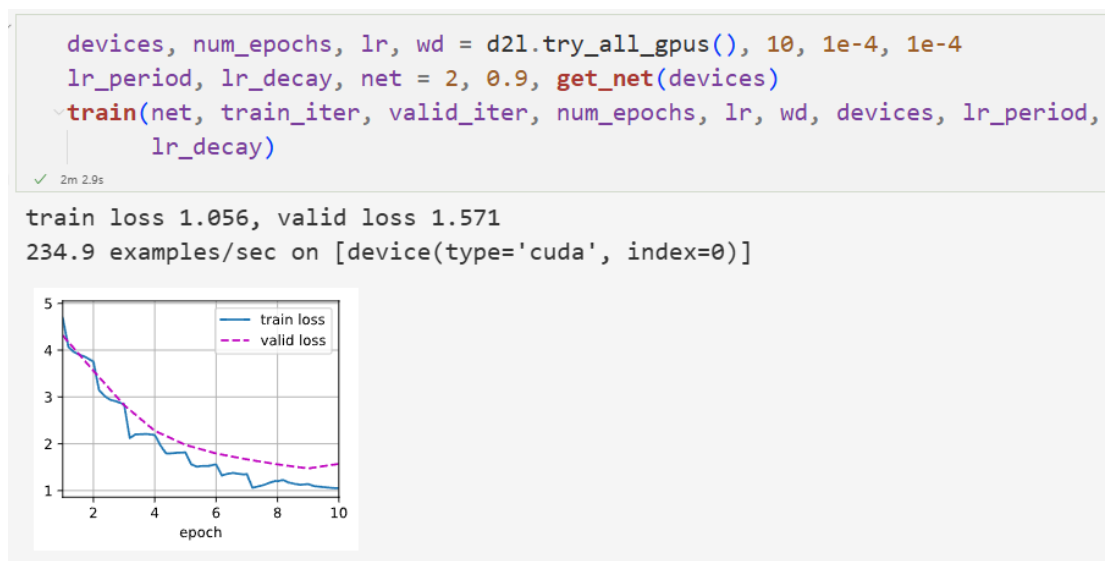
13.14.6 训练函数

根据模型在验证集上的表现选择模型并调整超参数。模型训练函数 train 只迭代小型自定义输出网络的参数。

```
def train(net, train_iter, valid_iter, num_epochs, lr, wd, devices, lr_period,
          lr_decay):
    # 只训练小型自定义输出网络
    net = nn.DataParallel(net, device_ids=devices).to(devices[0])
    trainer = torch.optim.SGD([param for param in net.parameters()
                                if param.requires_grad], lr=lr,
                               momentum=0.9, weight_decay=wd)
    scheduler = torch.optim.lr_scheduler.StepLR(trainer, lr_period, lr_decay)
    num_batches, timer = len(train_iter), d2l.Timer()
    legend = ['train loss']
    if valid_iter is not None:
        legend.append('valid loss')
    animator = d2l.Animator(xlabel='epoch', xlim=[1, num_epochs],
                           legend=legend)
    for epoch in range(num_epochs):
        metric = d2l.Accumulator(2)
        for i, (features, labels) in enumerate(train_iter):
            timer.start()
            features, labels = features.to(devices[0]), labels.to(devices[0])
            trainer.zero_grad()
            output = net(features)
            l = loss(output, labels).sum()
            l.backward()
            trainer.step()
            metric.add(l, labels.shape[0])
            timer.stop()
            if (i + 1) % (num_batches // 5) == 0 or i == num_batches - 1:
                animator.add(epoch + (i + 1) / num_batches,
                              (metric[0] / metric[1], None))
        measures = f'train loss {metric[0] / metric[1]:.3f}'
        if valid_iter is not None:
            valid_loss = evaluate_loss(valid_iter, net, devices)
            animator.add(epoch + 1, (None, valid_loss.detach().cpu()))
        scheduler.step()
    if valid_iter is not None:
        measures += f', valid loss {valid_loss:.3f}'
    print(measures + f'\n{metric[1] * num_epochs / timer.sum():.1f}'
          f' examples/sec on {str(devices)}')
```

13.14.7 训练和验证模型

训练和验证模型时，以下超参数都是可调的：可以增加迭代轮数。lr_period 和 lr_decay 分别设置为 2 和 0.9，优化算法的学习速率将在每 2 个迭代后乘以 0.9。



13.14.8 对测试集分类并在 Kaggle 提交结果

最终所有标记的数据（包括验证集）都用于训练模型和对测试集进行分类。使用训练好的自定义输出网络进行分类。



生成 scv 提交文件

A	B	C	D	E	F	G	H	I	J	K
id	affenpinscher	afghan_hound	african_hunting_dog	airedale	american_staffordshire_terrier	appenzeller	australian_terrier	basenji	basset	beagle
0dc570ec7086bab004a7e357164c04b8	0.02326628	1.02E-05	7.20E-05	0.001331321	3.37E-05	1.42E-05	0.09905545	0.000178799	3.01E-05	1
27ccf0e035abc33fb6a1e8bdf23c5f86	3.62E-05	0.003038362	8.00E-05	0.002870796	0.000188449	1.67E-05	1.37E-05	0.008982479	0.003314166	2
82d137d71f56cb0484b376ae3c867b14	5.40E-06	1.47E-05	9.46E-07	1.02E-06	4.42E-05	9.22E-05	2.13E-06	3.42E-05	0.007529881	3
8db94c3efef16b1d584ca43c4c8dea3b3	2.32E-05	6.31E-05	1.06E-07	8.41E-06	8.25E-06	1.05E-05	2.07E-07	2.09E-06	9.06E-06	4
b2f350d68b6264610c176e150dabc950	3.91E-05	0.000140007	0.000152168	0.003531031	0.014246225	0.000448156	0.000329079	0.004955872	0.000167162	0.00
b5cf42054fc30582086fd394c2c021	0.30930108	0.000338122	5.34E-05	0.000510982	0.000103717	7.33E-05	8.81E-05	0.000102966	1.57E-05	2
c76b5c3bda584485f913e25034c557d3	1.77E-05	0.000809827	3.34E-05	0.000335116	0.000210347	3.04E-05	9.99E-05	0.000653918	0.09052596	3
ec6b4aee97cf7eeb6cb26540af933db2	0.000160979	1.12E-05	7.06E-05	7.49E-05	0.011256686	0.001651822	0.000187531	0.051054776	0.023246676	0.0
f28b4d5899f9db2a2caa7a3361c847f1	4.70E-05	1.21E-07	0.00012539	0.002119773	2.65E-05	0.000306364	0.000151615	6.20E-05	4.22E-06	1
fd369fa622cf7437ddb4066a9d3ee534	0.0010110428	4.01E-05	0.000100752	0.000300567	0.008029656	0.000122136	0.002137506	0.001541816	0.002670379	0.00

实验 2：风格迁移，学习 13.12

滤波器能改变照片的颜色风格，从而使风景照更加锐利或者令人像更加美白。

但一个滤波器通常只能改变照片的某个方面。如果要照片达到理想中的风格，可能需要尝试大量不同的组合。

可以使用卷积神经网络，自动将一个图像中的风格应用在另一图像之上，即风格迁移，需要两张输入图像：一张是内容图像，另一张是风格图像。使用神经网络修改内容图像，使其在风格上接近风格图像。

方法

首先初始化合成图像，例如将其初始化为内容图像。该合成图像是风格迁移过程中唯一需要更新的变量，即风格迁移所需迭代的模型参数。

然后选择一个预训练的卷积神经网络来抽取图像的特征，其中的模型参数在训练中无须更新。这个深度卷积神经网络凭借多个层逐级抽取图像的特征，我们可以选择其中某些层的输出作为内容特征或风格特征

接下来通过前向传播（实线箭头方向）计算风格迁移的损失函数，并通过反向传播（虚线箭头方向）迭代模型参数，即不断更新合成图像。

风格迁移常用的损失函数由 3 部分组成：

1. 内容损失使合成图像与内容图像在内容特征上接近；
2. 风格损失使合成图像与风格图像在风格特征上接近；
3. 全变分损失则有助于减少合成图像中的噪点。

最后，当模型训练结束时输出风格迁移的模型参数，即得到最终的合成图像。

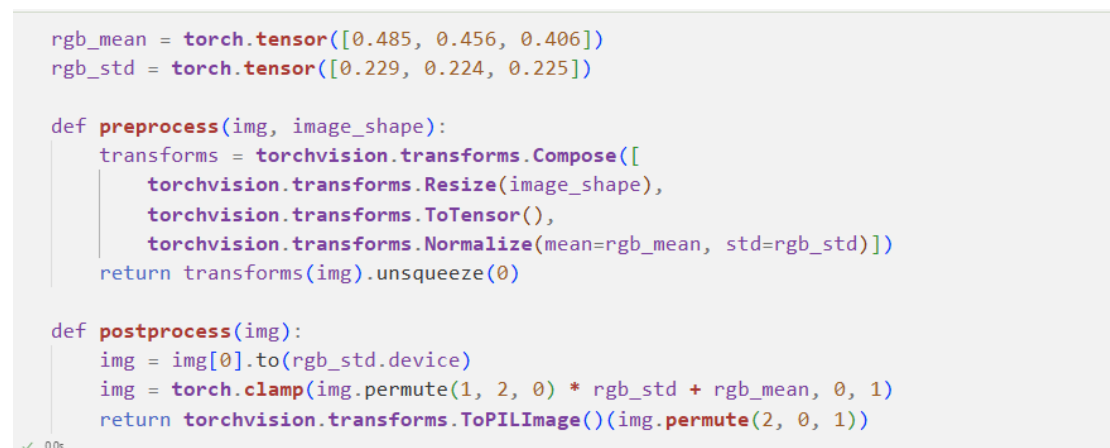
13.12.1 阅读内容和风格图像

首先读取内容和风格图像。从打印出的图像坐标轴可以看出，它们的尺寸并不一样。



13. 12. 2 预处理和后处理

定义图像的预处理函数和后处理函数：预处理函数 `preprocess` 对输入图像在 RGB 三个通道分别做标准化，并将结果变换成卷积神经网络接受的输入格式。后处理函数 `postprocess` 则将输出图像中的像素值还原回标准化之前的值。由于图像打印函数要求每个像素的浮点数值在 0~1 之间，所以对小于 0 和大于 1 的值分别取 0 和 1。



13. 12. 3 抽取图像特征

使用基于 ImageNet 数据集预训练的 VGG-19 模型来抽取图像特征

```
pretrained_net = torchvision.models.vgg19(pretrained=True)
```

2m 23.8s

d:\January\APP\anacanda3\envs\ndnd1gpu\lib\site-packages\torchvision\models\utils.py:208: UserWarning: The parameter 'pretrained' is deprecated, use 'weights' instead.
warnings.warn("The parameter 'pretrained' is deprecated, use 'weights' instead.")

d:\January\APP\anacanda3\envs\ndnd1gpu\lib\site-packages\torchvision\models\utils.py:223: UserWarning: Arguments other than a weight enum or a string are unsupported.
warnings.warn(msg)

Downloading: "https://download.pytorch.org/models/vgg19-dcbb9e9d.pth" to C:\Users\郭义月\.cache\torch\hub\checkpoints\vgg19-dcbb9e9d.pth
100.0%

为了抽取图像的内容特征和风格特征，我们可以选择 VGG 网络中某些层的输出。

一般来说，越靠近输入层，越容易抽取图像的细节信息；反之，则越容易抽取图像的全局信息。为了避免合成图像过多保留内容图像的细节，我们选择 VGG 较靠近输出的层，即内容层，来输出图像的内容特征。可以从 VGG 中选择不同层的输出来匹配局部和全局的风格，这些图层也称为风格层。

实验中选择第四卷积块的最后一个卷积层作为内容层，选择每个卷积块的一个卷积层作为风格层。这些层的索引可以通过打印 `pretrained_net` 实例获取。使用 VGG 层抽取特征时，只需要用到从输入层到最靠近输出层的内容层或风格层之间的所有层。下面构建一个新的网络 `net`，它只保留需要用到的 VGG 的所有层。

```
style_layers, content_layers = [0, 5, 10, 19, 28], [25]
```

```
net = nn.Sequential(*[pretrained_net.features[i] for i in  
                      range(max(content_layers + style_layers) + 1)])
```

给定输入 X ，需要中间层的输出，因此逐层计算，并保留内容层和风格层的输出。

```
def extract_features(X, content_layers, style_layers):
    contents = []
    styles = []
    for i in range(len(net)):
        X = net[i](X)
        if i in style_layers:
            styles.append(X)
        if i in content_layers:
            contents.append(X)
    return contents, styles
```

get_styles 函数对风格图像抽取风格特征。因为在训练时无须改变预训练的

VGG 的模型参数，所以可以在训练开始之前就提取出内容特征和风格特征。由于合成图像是风格迁移所需迭代的模型参数，只能在训练过程中通过调用 `extract_features` 函数来抽取合成图像的内容特征和风格特征。

```
def get_contents(image_shape, device):
    content_X = preprocess(content_img, image_shape).to(device)
    contents_Y, _ = extract_features(content_X, content_layers, style_layers)
    return content_X, contents_Y

def get_styles(image_shape, device):
    style_X = preprocess(style_img, image_shape).to(device)
    _, styles_Y = extract_features(style_X, content_layers, style_layers)
    return style_X, styles_Y
```

13.12.4 定义损失函数

风格迁移的损失函数由内容损失、风格损失和全变分损失 3 部分组成。

内容损失

与线性回归中的损失函数类似，内容损失通过平方误差函数衡量合成图像与内容图像在内容特征上的差异。平方误差函数的两个输入均为 `extract_features` 函数计算所得到的内容层的输出。

```
def content_loss(Y_hat, Y):
    # 我们从动态计算梯度的树中分离目标:
    # 这是一个规定的值, 而不是一个变量。
    return torch.square(Y_hat - Y.detach()).mean()
```

风格损失

风格损失与内容损失类似，也通过平方误差函数衡量合成图像与风格图像在风格上的差异。

为了表达风格层输出的风格先通过 `extract_features` 函数计算风格层的输出。

假设该输出的样本数为 1，通道数为 `c`，高和宽分别为 `h` 和 `w`，我们可以将此输出转换为矩阵 `X`，其有 `c` 行和 `hw` 列。

这个矩阵可以被看作由 `c` 个长度为 `hw` 的向量 $\mathbf{x}_1, \dots, \mathbf{x}_c$ 组合而成的。其中向量 $\mathbf{x}_i$ 代表了通道 `i` 上的风格特征。需要注意的是，当 `hw` 的值较大时，格拉姆矩阵中的元素容易出现较大的值。此外，格拉姆矩阵的高和宽皆为通道数 `c`。为了让风格损失不受这些值的大小影响，下面定义的 `gram` 函数将格拉姆矩阵除以了矩阵中元素的个数，即 `chw`。

```
def gram(X):
    num_channels, n = X.shape[1], X.numel() // X.shape[1]
    X = X.reshape((num_channels, n))
    return torch.matmul(X, X.T) / (num_channels * n)
```

风格损失的平方误差函数的两个格拉姆矩阵输入分别基于合成图像与风格图像的风格层输出。这里假设基于风格图像的格拉姆矩阵 `gram_Y` 已经预先计算好了。

```
def style_loss(Y_hat, gram_Y):
    return torch.square(gram(Y_hat) - gram_Y.detach()).mean()
```

全变分损失

有时候学到的合成图像里面有大量高频噪点，即有特别亮或者特别暗的颗粒像素。一种常见的去噪方法是全变分去噪

```
def tv_loss(Y_hat):
    return 0.5 * (torch.abs(Y_hat[:, :, 1:, :] - Y_hat[:, :, :-1, :]).mean() +
                  torch.abs(Y_hat[:, :, :, 1:] - Y_hat[:, :, :, :-1]).mean())
```

损失函数

风格转移的损失函数是内容损失、风格损失和总变化损失的加权和。

通过调节这些权重超参数，我们可以权衡合成图像在保留内容、迁移风格以及去噪三方面的相对重要性。

```
content_weight, style_weight, tv_weight = 1, 1e3, 10

def compute_loss(X, contents_Y_hat, styles_Y_hat, contents_Y, styles_Y_gram):
    # 分别计算内容损失、风格损失和全变分损失
    contents_l = [content_loss(Y_hat, Y) * content_weight for Y_hat, Y in zip(
        contents_Y_hat, contents_Y)]
    styles_l = [style_loss(Y_hat, Y) * style_weight for Y_hat, Y in zip(
        styles_Y_hat, styles_Y_gram)]
    tv_l = tv_loss(X) * tv_weight
    # 对所有损失求和
    l = sum(10 * styles_l + contents_l + [tv_l])
    return contents_l, styles_l, tv_l, l
```

13.12.5 初始化合成图像

在风格迁移中，合成的图像是训练期间唯一需要更新的变量。因此可以定义一个简单的模型 `SynthesizedImage`，并将合成的图像视为模型参数。模型的前向传播只需返回模型参数即可。

```
class SynthesizedImage(nn.Module):
    def __init__(self, img_shape, **kwargs):
        super(SynthesizedImage, self).__init__(**kwargs)
        self.weight = nn.Parameter(torch.rand(*img_shape))

    def forward(self):
        return self.weight
```

定义 `get_inits` 函数。该函数创建了合成图像的模型实例，并将其初始化为图像 `X`。风格图像在各个风格层的格拉姆矩阵 `styles_Y_gram` 将在训练前预先计算好。

```
def get_inits(X, device, lr, styles_Y):
    gen_img = SynthesizedImage(X.shape).to(device)
    gen_img.weight.data.copy_(X.data)
    trainer = torch.optim.Adam(gen_img.parameters(), lr=lr)
    styles_Y_gram = [gram(Y) for Y in styles_Y]
    return gen_img(), styles_Y_gram, trainer
```

13.12.6 训练模型

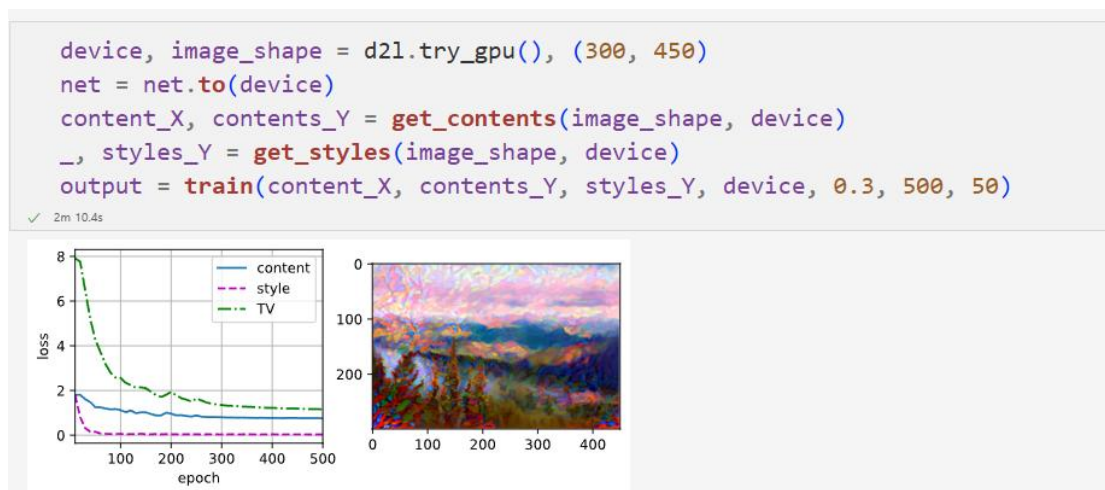
在训练模型进行风格迁移时不断抽取合成图像的内容特征和风格特征，然后计算损失函数。下面定义了训练循环。

```
def train(X, contents_Y, styles_Y, device, lr, num_epochs, lr_decay_epoch):
    X, styles_Y_gram, trainer = get_inits(X, device, lr, styles_Y)
    scheduler = torch.optim.lr_scheduler.StepLR(trainer, lr_decay_epoch, 0.8)
    animator = d2l.Animator(xlabel='epoch', ylabel='loss',
                           xlim=[10, num_epochs],
                           legend=['content', 'style', 'TV'],
                           ncols=2, figsize=(7, 2.5))

    for epoch in range(num_epochs):
        trainer.zero_grad()
        contents_Y_hat, styles_Y_hat = extract_features(
            X, content_layers, style_layers)
        contents_l, styles_l, tv_l, l = compute_loss(
            X, contents_Y_hat, styles_Y_hat, contents_Y, styles_Y_gram)
        l.backward()
        trainer.step()
        scheduler.step()
        if (epoch + 1) % 10 == 0:
            animator.axes[1].imshow(postprocess(X))
            animator.add(epoch + 1, [float(sum(contents_l)),
                                     float(sum(styles_l)), float(tv_l)])

    return X
```

训练模型：首先将内容图像和风格图像的高和宽分别调整为 300 和 450 像素，用内容图像来初始化合成图像。



可以看到，合成图像保留了内容图像的风景和物体，并同时迁移了风格图像的色彩。例如，合成图像具有与风格图像中一样的色彩块，其中一些甚至具有画笔笔触的细微纹理

实验 3：语义分割与数据集，学习 13.9

语义分割重点关注于如何将图像分割成属于不同语义类别的区域。与目标检测不同，语义分割可以识别并理解图像中每一个像素的内容：其语义区域的标注和预测是像素级的。与目标检测相比，语义分割标注的像素级的边框显然更加精细。计算机视觉领域还有 2 个与语义分割相似的重要问题，即图像分割和实例分割。图像分割将图像划分为若干组成区域，这类问题的方法通常利用图像中像素之间的相关性。它在训练时不需要有关图像像素的标签信息，在预测时也无法保证分割出的区域具有我们希望得到的语义。以 `numrefig_segmentation` 中的图像作为输入，图像分割可能会将狗分为两个区域：一个覆盖以黑色为主的嘴和眼睛，另一个覆盖以黄色为主的其余部分身体。

实例分割也叫同时检测并分割，它研究如何识别图像中各个目标实例的像素级区域。与语义分割不同，实例分割不仅需要区分语义，还要区分不同的目标实例。例如，如果图像中有两条狗，则实例分割需要区分像素属于的两条狗中的哪一条。

13.9.1 PascalVOC2012 语义分割数据集

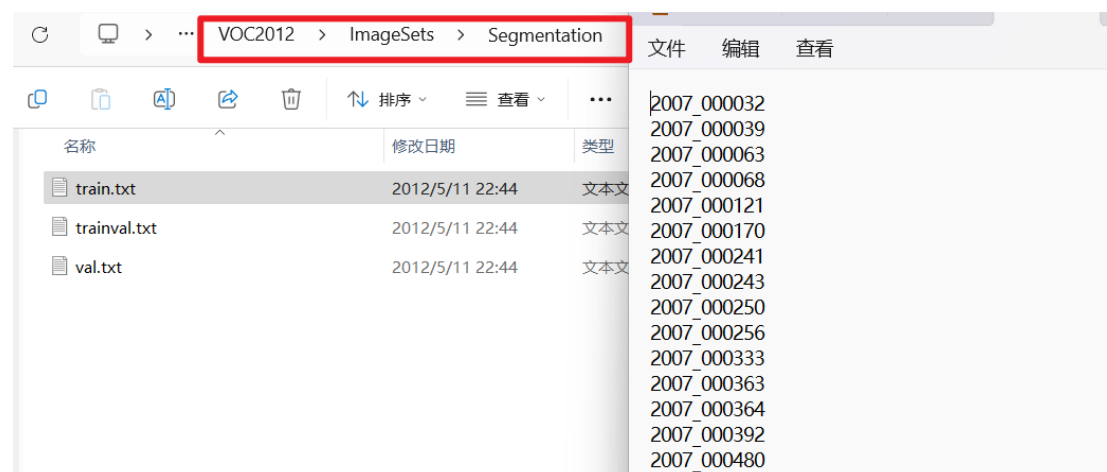
```
##save
d2l.DATA_HUB['voc2012'] = (d2l.DATA_URL + 'VOCtrainval_11-May-2012.tar',
                           '4e443f8a2eca6b1dac8a6c57641b67dd40621a49')

voc_dir = d2l.download_extract('voc2012', 'VOCdevkit/VOC2012')
```

✓ 7m 25.4s

Downloading ../data/VOCtrainval_11-May-2012.tar from http://d2l-data.s3-accelerate.amazonaws.com/VOCtrainval_11-May-2012.tar...

进入路径之后可以看到数据集的不同组件。ImageSets/Segmentation 路径包含用于训练和测试样本的文本文件，而 JPEGImages 和 SegmentationClass 路径分别存储着每个示例的输入图像和标签。此处的标签也采用图像格式，其尺寸和它所标注的输入图像的尺寸相同。此外，标签中颜色相同的像素属于同一个语义类别。

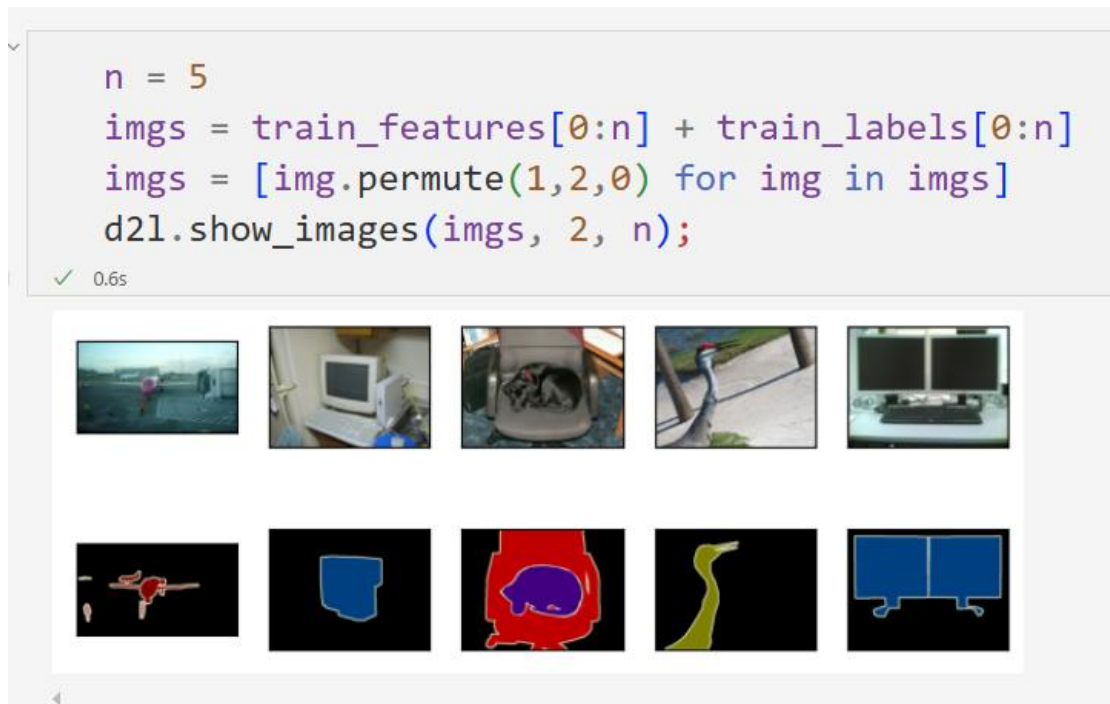


read_voc_images 函数：将所有输入的图像和标签读入内存。

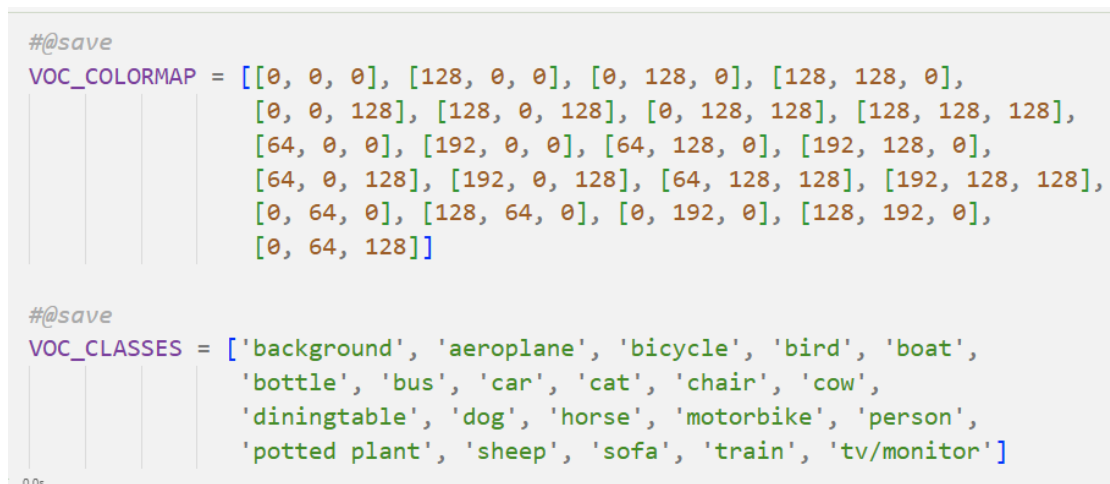
```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
#@save
def read_voc_images(voc_dir, is_train=True):
    """读取所有VOC图像并标注"""
    txt_fname = os.path.join(voc_dir, 'ImageSets', 'Segmentation',
                              'train.txt' if is_train else 'val.txt')
    mode = torchvision.io.image.ImageReadMode.RGB
    with open(txt_fname, 'r') as f:
        images = f.read().split()
        features, labels = [], []
        for i, fname in enumerate(images):
            features.append(torchvision.io.read_image(os.path.join(
                voc_dir, 'JPEGImages', f'{fname}.jpg')))
            labels.append(torchvision.io.read_image(os.path.join(
                voc_dir, 'SegmentationClass', f'{fname}.png'), mode))
        return features, labels

train_features, train_labels = read_voc_images(voc_dir, True)
```

绘制前 5 个输入图像及其标签，在标签图像中，白色和黑色分别表示边框和背景，而其他颜色则对应不同的类别。



列举 RGB 颜色值和类名



通过上面定义的两个常量可以方便地查找标签中每个像素的类索引。

`voc_colormap2label` 函数：构建从上述 RGB 颜色值到类别索引的映射

`voc_label_indices` 函数：将 RGB 值映射到在 Pascal VOC2012 数据集中的类别索引。


```

#@save
def voc_colormap2label():
    """构建从RGB到VOC类别索引的映射"""
    colormap2label = torch.zeros(256 ** 3, dtype=torch.long)
    for i, colormap in enumerate(VOC_COLORMAP):
        colormap2label[
            (colormap[0] * 256 + colormap[1]) * 256 + colormap[2]] = i
    return colormap2label

#@save
def voc_label_indices(colormap, colormap2label):
    """将VOC标签中的RGB值映射到它们的类别索引"""
    colormap = colormap.permute(1, 2, 0).numpy().astype('int32')
    idx = ((colormap[:, :, 0] * 256 + colormap[:, :, 1]) * 256
           + colormap[:, :, 2])
    return colormap2label[idx]

```

在第一张样本图像中，飞机头部区域的类别索引为 1，而背景索引为 0。

```

y = voc_label_indices(train_labels[0], voc_colormap2label())
y[105:115, 130:140], VOC_CLASSES[1]

```

✓ 0.0s

```

(tensor([[0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
        [0, 0, 0, 0, 0, 0, 0, 1, 1, 1],
        [0, 0, 0, 0, 0, 0, 1, 1, 1, 1],
        [0, 0, 0, 0, 0, 1, 1, 1, 1, 1],
        [0, 0, 0, 0, 0, 1, 1, 1, 1, 1],
        [0, 0, 0, 0, 1, 1, 1, 1, 1, 1],
        [0, 0, 0, 0, 0, 1, 1, 1, 1, 1],
        [0, 0, 0, 0, 0, 1, 1, 1, 1, 1],
        [0, 0, 0, 0, 0, 0, 1, 1, 1, 1],
        [0, 0, 0, 0, 0, 0, 0, 1, 1, 1]]),
'aeroplane')

```

13.9.2 预处理数据

将图像裁剪为固定尺寸，而不是再缩放。具体来说使用图像增广中的随机裁剪，裁剪输入图像和标签的相同区域。

```

#@save
def voc_rand_crop(feature, label, height, width):
    """随机裁剪特征和标签图像"""
    rect = torchvision.transforms.RandomCrop.get_params(
        feature, (height, width))
    feature = torchvision.transforms.functional.crop(feature, *rect)
    label = torchvision.transforms.functional.crop(label, *rect)
    return feature, label

```

✓ 0.0s

```
imgs = []
for _ in range(n):
    imgs += voc_rand_crop(train_features[0], train_labels[0], 200, 300)

imgs = [img.permute(1, 2, 0) for img in imgs]
d2l.show_images(imgs[::2] + imgs[1::2], 2, n);
```

✓ 0.4s



13.9.3 自定义语义分割数据集类

通过继承高级 API 提供的 Dataset 类，自定义了一个语义分割数据集

VOCSegDataset。通过实现__getitem__函数，可以任意访问数据集中索引为 idx 的输入图像及其每个像素的类别索引。

由于数据集中有些图像的尺寸可能小于随机裁剪所指定的输出尺寸，这些样本可以通过自定义的`filter`函数移除掉。

此外，还定义了 normalize_image 函数，从而对输入图像的 RGB 三个通道的值分别做标准化。

```
##@save
class VOCSegDataset(torch.utils.data.Dataset):
    """一个用于加载VOC数据集的自定义数据集"""

    def __init__(self, is_train, crop_size, voc_dir):
        self.transform = torchvision.transforms.Normalize(
            mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
        self.crop_size = crop_size
        features, labels = read_voc_images(voc_dir, is_train=is_train)
        self.features = [self.normalize_image(feature)
                        for feature in self.filter(features)]
        self.labels = self.filter(labels)
        self.colormap2label = voc_colormap2label()
        print('read ' + str(len(self.features)) + ' examples')

    def normalize_image(self, img):
        return self.transform(img.float() / 255)

    def filter(self, imgs):
        return [img for img in imgs if (
            img.shape[1] >= self.crop_size[0] and
            img.shape[2] >= self.crop_size[1])]

    def __getitem__(self, idx):
        feature, label = voc_rand_crop(self.features[idx], self.labels[idx],
                                      *self.crop_size)
        return (feature, voc_label_indices(label, self.colormap2label))

    def __len__(self):
        return len(self.features)
```

✓ 0.0s

13.9.4 读取数据集

通过自定义的 `VOCSegDataset` 类来分别创建训练集和测试集的实例。指定随机裁剪的输出图像的形狀为 $320 * 480$ ，可以查看训练集和测试集所保留的样本个数。

```
crop_size = (320, 480)
voc_train = VOCSegDataset(True, crop_size, voc_dir)
voc_test = VOCSegDataset(False, crop_size, voc_dir)
```

✓ 50.6s

```
read 1114 examples
read 1078 examples
```

设批量大小为 64 定义训练集的迭代器。打印第一个小批量的形状会发现：与图像分类或目标检测不同，这里的标签是一个三维数组。

```
batch_size = 64
train_iter = torch.utils.data.DataLoader(voc_train, batch_size, shuffle=True,
                                         drop_last=True,
                                         num_workers=d2l.get_dataloader_workers())

for X, Y in train_iter:
    print(X.shape)
    print(Y.shape)
    break
```

13.9.5 整合所有组件

定义 `load_data_voc` 函数来下载并读取 Pascal VOC2012 语义分割数据集。它返回训练集和测试集的数据迭代器。

```
##@save
def load_data_voc(batch_size, crop_size):
    """加载VOC语义分割数据集"""
    voc_dir = d2l.download_extract('voc2012', os.path.join(
        'VOCdevkit', 'VOC2012'))
    num_workers = d2l.get_dataloader_workers()
    train_iter = torch.utils.data.DataLoader(
        VOCSegDataset(True, crop_size, voc_dir), batch_size,
        shuffle=True, drop_last=True, num_workers=num_workers)
    test_iter = torch.utils.data.DataLoader(
        VOCSegDataset(False, crop_size, voc_dir), batch_size,
        drop_last=True, num_workers=num_workers)
    return train_iter, test_iter
```

✓ 0.0s

[小结或讨论]

习题

13.14 习题 2

如果使用更深的预训练模型，会得到更好的结果吗？如何调整超参数？能进一步改善结果吗？

使用更深的训练模型有以下优势：

特征表达能力更强：更深的模型（如 ResNet-50/101/152）拥有更多卷积层和参数，能捕捉更抽象的视觉特征（如狗的毛发纹理、面部结构等细节）。例如，ResNet-50 比 ResNet-34 多了约 2 倍参数，可学习更复杂的品种判别特征。

层次化特征提取：深层网络通过多层非线性变换，能从像素级边缘逐步提取到语义级品种特征。例如，浅层网络关注颜色和轮廓，深层网络关注“垂耳”“卷毛”等品种特异性特征。

但是也可能有以下劣势：

过拟合风险：数据集仅 120 类、约 1 万张训练图，深层模型易对训练数据过拟合（如记住特定个体特征而非品种共性）。

计算资源消耗：ResNet-101 的参数量是 ResNet-34 的 3 倍以上，训练时内存需求更高，推理速度更慢。

在超参数调整方面，需根据更深预训练模型的特性进行针对性优化。学习率方面，由于深层模型参数更多，为避免梯度爆炸，应将学习率降低至 $1e-5$ 到 $5e-5$ 区间，例如 ResNet-50 可设为 $5e-5$ ，并每 10 轮按 0.1 倍衰减。权重衰减需相应增加到 $1e-3$ 至 $5e-3$ ，以此增强正则化效果，有效抑制过拟合问题。

训练轮数要适当增加，一般设为 20 到 30 轮，但需结合验证集损失采用早停策略，防止训练时间过长导致过拟合。批次大小若内存允许可增大到 64 至 128，这样能使梯度更稳定，不过调整批次大小的同时要同步调整学习率。

关于模型冻结层数，可仅解冻最后 1 到 2 个残差块，其余层保持冻结状态，避免大量参数同时更新给优化过程带来困难。比如对于 ResNet-50，可只解冻 layer4 和输出层，其他层继续冻结。数据增强方面，增加旋转、缩放、弹性变换等强增强策略，能够扩充训练数据的多样性，帮助模型更好地学习品种特征。

更深的模型：

```
def get_deep_net(devices):
    # 替换为更深的预训练模型（如ResNet-50）
    finetune_net = nn.Sequential()
    finetune_net.features = torchvision.models.resnet50(pretrained=True)
    # 调整输出层以匹配120类
    finetune_net.output_new = nn.Sequential(
        nn.Linear(2048, 512), # 深层模型特征维度更高（ResNet-50输出2048维）
        nn.ReLU(),
        nn.Dropout(0.5), # 增加Dropout抑制过拟合
        nn.Linear(512, 120)
    )
    # 冻结参数（仅解冻最后几层）
    for name, param in finetune_net.features.named_parameters():
        # 仅解冻最后一个残差块（layer4）和后续层
        if not name.startswith('layer4'):
            param.requires_grad = False
    return finetune_net.to(devices[0])
```

还可以分阶段训练，先冻结所有预训练层，仅训练输出网络 10 轮，让模型适应品种分类任务；解冻最后 1~2 个残差块，降低学习率至 $1e-5$ ，再训练 10~15 轮，微调高层特征。

13.12 习题 1

选择不同的内容和风格层，输出有什么变化？

选择其他风格的图片进行更换：

The Jupyter Notebook code and output are as follows:

```
import torchvision
from torch import nn
from d2l import torch as d2l

d2l.set_figsize()
content_img = d2l.Image.open('../img/waldo-mask.jpg')
d2l.plt.imshow(content_img);

style_img = d2l.Image.open('../img/koebel.jpg')
d2l.plt.imshow(style_img);
```

The file explorer shows the following files:

名称	修改日期	类型
catdog.jpg	2025/6/11 20:36	JPG 文件
death-cap.jpg	2025/6/11 20:36	JPG 文件
deeplearning-amazon.jpg	2025/6/11 20:36	JPG 文件
dog1.jpg	2025/6/11 20:36	JPG 文件
dog2.jpg	2025/6/11 20:36	JPG 文件
kaggle-dog.jpg	2025/6/11 20:36	JPG 文件
koebel.jpg	2025/6/11 20:36	JPG 文件
neural-style.jpg	2025/6/11 20:36	JPG 文件
pikachu.jpg	2025/6/11 20:36	JPG 文件
rainier.jpg	2025/6/11 20:36	JPG 文件
tensorflow.jpg	2025/6/11 20:36	JPG 文件
waldo-mask.jpg	2025/6/11 20:36	JPG 文件
where-wally-walker-books.jpg	2025/6/11 20:36	JPG 文件
a77.svg	2025/6/11 20:36	Microsoft
add_norm.svg	2025/6/11 20:36	Microsoft
alexnet.svg	2025/6/11 20:36	Microsoft

训练结果为：

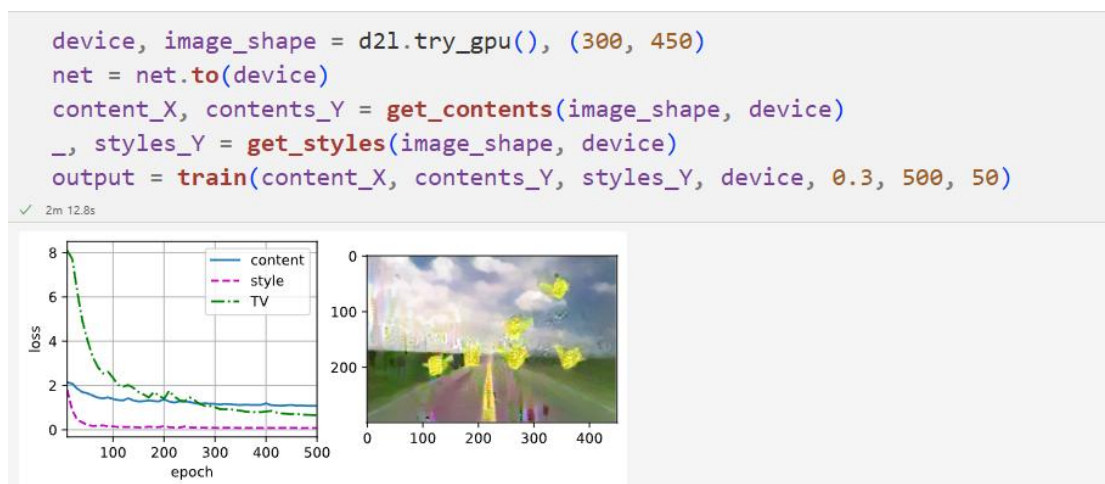


可以观察到，虽然结果比较奇怪，但是可以顺利实现图像风格迁移

如果更换成这两张图片：



实验结果为：



可以看到，两张图片完全不相干，结果变得更加奇怪了，但是还是有风格迁移的效果的

13.12 习题 2

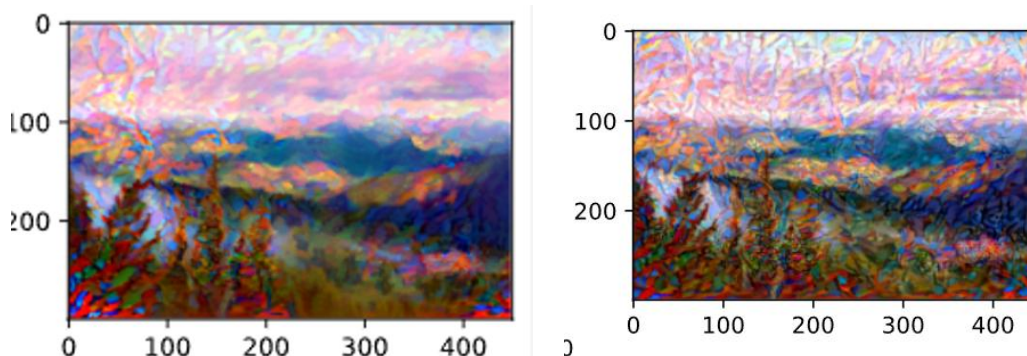
调整损失函数中的权重超参数。输出是否保留更多内容或减少更多噪点？

增大风格损失函数权重：



结果：

左图是原来的参数，右图是增大风格损失函数权重后：



减小内容损失函数：


```

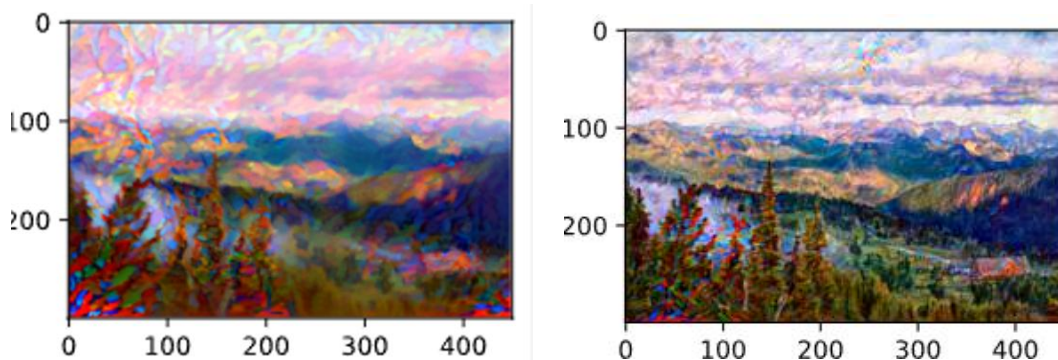
content_weight, style_weight, tv_weight = 1, 1e3, 10

def compute_loss(X, contents_Y_hat, styles_Y_hat, contents_Y, styles_Y_gram):
    # 分别计算内容损失、风格损失和全变分损失
    contents_l = [content_loss(Y_hat, Y) * content_weight for Y_hat, Y in zip(
        contents_Y_hat, contents_Y)]
    styles_l = [style_loss(Y_hat, Y) * style_weight for Y_hat, Y in zip(
        styles_Y_hat, styles_Y_gram)]
    tv_l = tv_loss(X) * tv_weight
    # 对所有损失求和
    # l = sum(10 * styles_l + contents_l + [tv_l])
    l = sum(10 * styles_l + 0.1 * contents_l + [tv_l])
    return contents_l, styles_l, tv_l, l

```

减小内容损失函数

实验结果：左图是原来的参数，右图是减小内容损失函数权重后：



减小全局变损失函数权重：

```

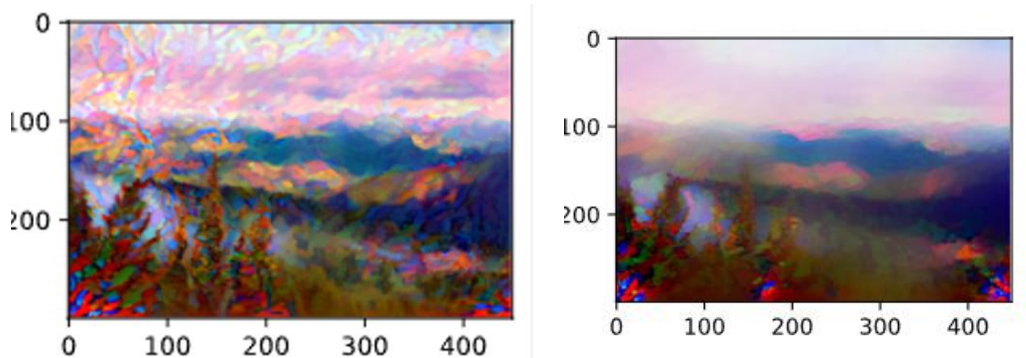
content_weight, style_weight, tv_weight = 1, 1e3, 10

def compute_loss(X, contents_Y_hat, styles_Y_hat, contents_Y, styles_Y_gram):
    # 分别计算内容损失、风格损失和全变分损失
    contents_l = [content_loss(Y_hat, Y) * content_weight for Y_hat, Y in zip(
        contents_Y_hat, contents_Y)]
    styles_l = [style_loss(Y_hat, Y) * style_weight for Y_hat, Y in zip(
        styles_Y_hat, styles_Y_gram)]
    tv_l = tv_loss(X) * tv_weight
    # 对所有损失求和
    # l = sum(10 * styles_l + contents_l + [tv_l])
    l = sum(10 * styles_l + contents_l + 0.1 * [tv_l])
    return contents_l, styles_l, tv_l, l

```

减少全变分损失权重

实验结果：左图是原来的参数，右图是减小变损失函数权重后：



实验结果发现，当调整参数时，风格迁移的结果肉眼可见的显著的变换，通过理论分析可知：

内容损失权重增大时，合成图像更保留内容图像的主体结构（如物体形状、轮廓），风格迁移效果减弱；减小时，风格纹理会更强烈地覆盖内容，可能导致内容扭曲。

风格损失权重增大时，合成图像的风格特征（如油画笔触、色彩分布）更明显，但内容清晰度可能下降；减小时，风格影响减弱，内容原始细节更突出。

全变分损失权重增大时，合成图像噪点减少、像素过渡更平滑，但可能模糊细节；减小时，细节更丰富，但易出现颗粒感或噪点。

平衡策略：若需保留内容，可增大内容损失权重和全变分损失权重；若追求风格强度，可提升风格损失权重并配合全变分损失权重去噪；默认权重（1:1e3:10）适用于多数场景，微调时建议按比例调整以避免优化失衡。

调参数的时候找到三者的一个平衡是最重要的

13.9 习题 1

如何在自动驾驶和医疗图像诊断中应用语义分割？还能想到其他领域的应用吗？

语义分割作为计算机视觉中像素级分类任务，在自动驾驶、医疗诊断等领域有重要应用，同时也广泛渗透至其他行业。

1. 自动驾驶中的语义分割应用

环境实时理解：通过分割道路、车辆、行人、交通标志等语义单元，为自动驾驶提供决策依据。例如，利用 DeepLab 系列模型对车载摄像头图像进行实时分割，识别可行驶区域（道路）与障碍物（行人、车辆），精度需达 95% 以上以保障行车安全。**动态障碍物预测：**结合时序语义分割结果（如连续帧中车辆的位置变化），可预测物体运动轨迹。如 NVIDIA 的 DriveNet 通过多任务网络同时实现语义分割与运动预测，提升自动驾驶系统的反应速度。

2. 医疗图像诊断中的语义分割应用

病灶精准定位：在肺部 CT 中分割结节、乳腺 X 光中识别肿瘤区域，辅助医生量化病灶大小与形状。例如，SegNet 在肺部结节分割中，Dice 系数可达 90%，帮助早期肺癌筛查。**手术规划与预后评估：**通过分割器官结构（如肝脏、脑部灰质），为微创手术提供三维重建基础。如 3DUNet 在脑部 MRI 分割中，可精确划分肿瘤与正常组织边界，指导手术路径规划。

3. 其他领域的语义分割应用拓展

农业与智慧 farming

作物健康监测：通过分割农田图像中的作物与杂草，自动喷洒农药。如基于 UNet 的模型在玉米田分割中，可区分 90% 以上的作物与杂草，减少 30% 农药使用量。

产量预估：分割果实与枝叶，统计挂果数量。例如，华为 HiLens 平台的果园分割方案，通过 YOLO 与语义分割结合，实现柑橘产量的自动化估算。

工业质检与缺陷检测

表面缺陷分割：在半导体晶圆、汽车零部件图像中，分割划痕、裂纹等缺陷区域。

如富士康采用 DeepLabv3+ 模型，对 PCB 电路板缺陷分割的准确率达 98%，替代人工目检。物料分拣：分割传送带上的不同物料，指导机械臂抓取。如亚马逊物流中心利用语义分割与 3D 点云融合，实现包裹类型的实时分类与分拣。

安防与智慧城市

异常行为监测：在监控视频中分割异常行为区域（如翻越围栏、人群聚集）。如海康威视的行为分析系统，通过时空语义分割，识别暴力行为的准确率达 85%。

城市基础设施管理：分割道路裂缝、井盖缺失等隐患，如市政部门利用无人机图像分割，自动定位破损路面，维修效率提升 40%。

4. 跨领域应用的共性挑战与趋势

1. 算力与部署成本：边缘设备（如自动驾驶车载芯片、医疗便携设备）需轻量化模型，TensorRT 等推理优化框架成为刚需。

2. 多模态融合：单一图像分割难以应对复杂场景，未来需结合激光雷达、红外等多源数据，如特斯拉 FSD 的多传感器语义分割方案。

3. 伦理与安全：医疗分割中的患者隐私保护、自动驾驶中的决策安全性，推动联邦学习、差分隐私等技术与语义分割的结合。

语义分割的跨领域应用本质是“从像素理解到场景决策”的升级，其技术演进需同时兼顾精度、效率与场景适配性，未来在 AIoT、元宇宙等领域将有更深度的融合。

讨论与收获

1. 视觉分类的迁移学习实践

通过微调 ResNet-34 实现狗品种分类，验证了预训练模型在特征提取上的优势。

冻结底层参数仅训练输出层的策略有效降低计算成本，数据增强处理（如随机裁剪、色彩抖动）显著提升模型泛化能力。理论上，更深层模型（如 ResNet-50）可通过分阶段微调优化复杂品种区分度，但需关注过拟合风险，这为实际场景中模型选型提供了理论依据。

2. 风格迁移的损失函数调控

实验表明，损失函数权重直接影响合成图像的内容保留与风格强度。内容权重增大时主体结构更清晰，风格权重提升至 $1e4$ 会强化艺术纹理但可能扭曲细节，全变分损失能有效抑制噪点但过度使用会模糊纹理。默认权重（1:1e3:10）的平衡策略适用于多数场景，参数微调需按比例调整以避免优化失衡，这为实际风格迁移任务提供了参数调节的理论框架。

3. 语义分割的跨领域应用价值

语义分割在自动驾驶（道路与障碍物实时分割）和医疗诊断（病灶精准定位）中的应用，体现了像素级理解的技术价值。农作物分割、工业缺陷检测等拓展场景表明，模型轻量化与多模态融合是突破复杂环境限制的关键。实验通过理论推导与简化验证，明确了不同领域任务中模型设计与参数优化的核心方向。